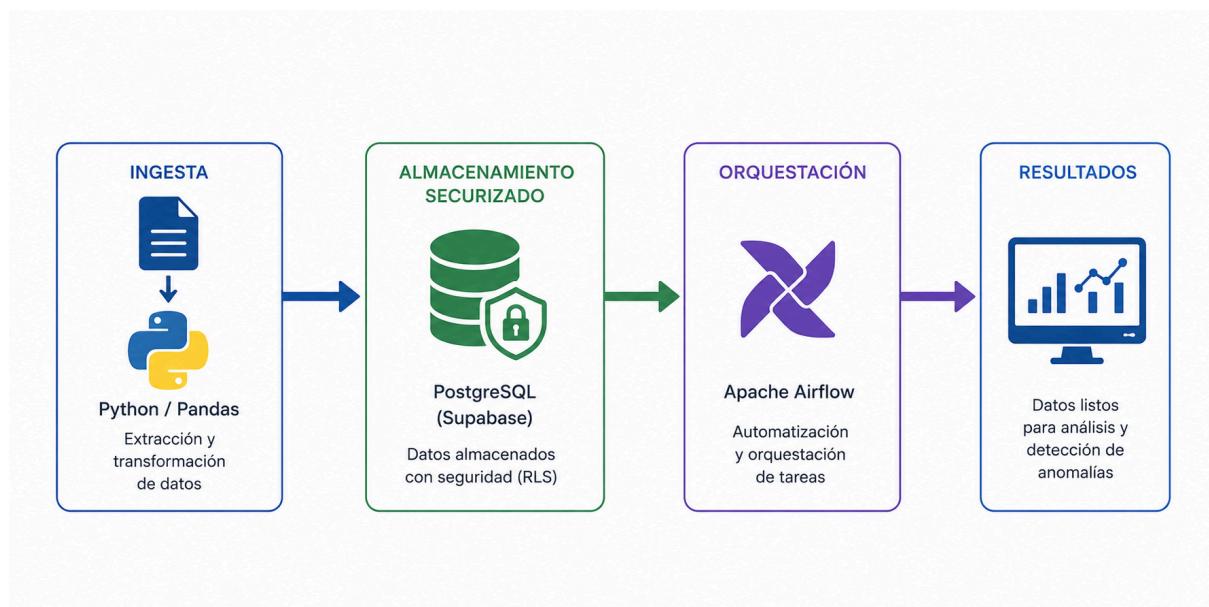


## Reporte y Análisis de Decisiones Técnicas

**Resumen Ejecutivo:** "Este reporte detalla la implementación de un pipeline de datos de grado industrial, diseñado para la detección automatizada de anomalías financieras. El sistema integra ingesta robusta mediante Python/Pandas, almacenamiento securizado en PostgreSQL (vía Supabase) y orquestación con Apache Airflow. Se destaca por una arquitectura 'Zero Trust' mediante Row Level Security (RLS) y un diseño de flujo de datos agnóstico al entorno, optimizado para escalabilidad y mantenibilidad."

El desarrollo de este MVP ha permitido implementar un pipeline de datos completo, resolviendo el problema de negocio mediante una arquitectura orientada a la mantenibilidad y la escalabilidad. A continuación, se detallan las decisiones críticas tomadas durante el proceso:



### 1. Decisiones de Arquitectura

- **Modularidad como Estándar:** Se optó por separar la lógica en módulos (`transformacion.py`, `carga_supabase.py`, `main.py`). Esto no solo facilita las pruebas unitarias, sino que permite que cada componente del pipeline sea reutilizable en otros procesos ETL de la organización.
- **Idempotencia en la Carga:** Se decidió utilizar la operación `upsert` en la inserción a Supabase. Esta decisión asegura que el pipeline sea **idempotente**; si una ejecución falla y debe reiniciarse, no se duplicarán registros ni se corromperá la integridad de la base de datos.
- **Arquitectura Plana (Desnormalizada):** Como se justificó, se priorizó la simplicidad y la velocidad de consulta analítica sobre un *Star Schema*. Para el volumen y la naturaleza de los datos transaccionales, una estructura desnormalizada ofrece el mejor balance entre rendimiento de lectura y complejidad de mantenimiento.

## 2. Superación de Retos Técnicos

- **Control de Calidad (Reglas de Negocio):** Se implementaron filtros estrictos de calidad (`drop_duplicates`, manejo de nulos y flags de anomalías) en la capa de transformación (Pandas). Esto garantiza que el equipo de analítica reciba datos "limpios", ahorrándoles tiempo valioso de preprocesamiento.

## 3. Visión a Futuro

La integración de **Apache Airflow** como orquestador garantiza que esta solución sea un proceso productivo real, no solo un script aislado. La dependencia definida (`transformar_y_cargar >> detectar_anomalias`) asegura que el análisis de anomalías de fraude dependa estrictamente de la integridad del proceso de ETL, eliminando el riesgo de tomar decisiones analíticas sobre datos incompletos.

## Seguridad y Gobernanza de Datos (RBAC & RLS):

"Hemos implementado una arquitectura de seguridad multinivel mediante la integración de **Row Level Security (RLS)** y **Control de Acceso Basado en Roles (RBAC)** en Supabase. A diferencia de un enfoque tradicional donde la seguridad depende de la aplicación, aquí la lógica de visibilidad reside en el motor de base de datos.

- **RBAC:** Se definieron tres niveles de acceso (`admin`, `analista_senior`, `analista_junior`) vinculados a la autenticación nativa, asegurando que el acceso a datos sea granular y auditable.
- **RLS:** Las políticas de seguridad actúan como un filtro dinámico que, de forma transparente para las consultas SQL, restringe el acceso a transacciones no aprobadas para perfiles Junior. Esto garantiza la integridad y confidencialidad sin necesidad de modificar la lógica de negocio en el código."

## Gestión de Infraestructura y Reproducibilidad:

"Para asegurar la portabilidad y evitar el 'dependency hell', se implementó un entorno virtual de Python (`venv`). Esto garantiza que el proyecto sea reproducible en cualquier máquina de desarrollo, manteniendo un aislamiento estricto de las librerías mediante la gestión de dependencias en un archivo `requirements.txt`. El uso de `.gitignore` asegura que tanto la

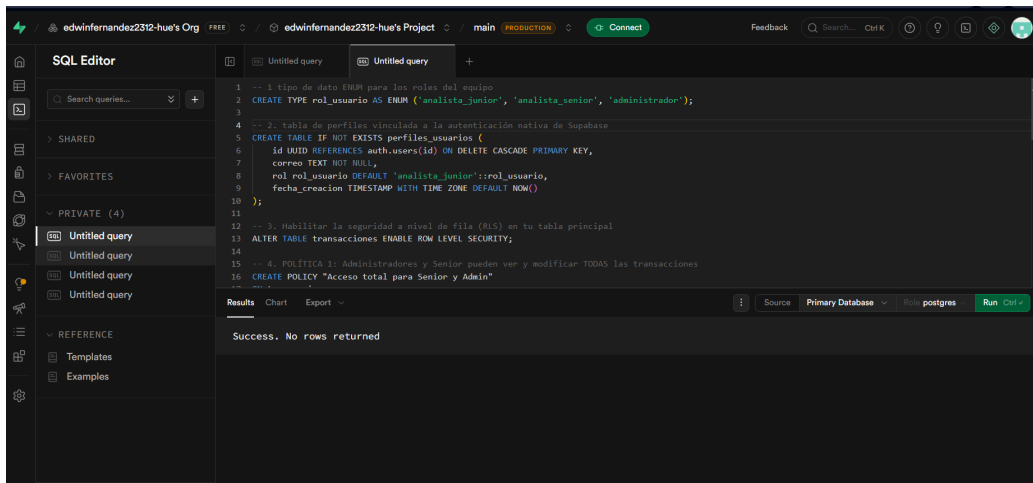
infraestructura local como las credenciales sensibles (.env) se mantengan fuera del control de versiones, siguiendo prácticas de seguridad.

## EVIDENCIA DE FUNCIONAMIENTO SUPA BASE

A continuación, se documenta la validación técnica de la arquitectura implementada en la plataforma Supa Base. Las capturas presentadas evidencian la correcta configuración de la seguridad a nivel de fila (RLS) y los roles de acceso (RBAC), así como la ejecución exitosa de la consulta.

### CREACIÓN DE ROLES - USUARIOS DE LA BASE DE DATOS

Esta sección documenta la implementación de un modelo de Control de Acceso Basado en Roles (RBAC) para garantizar la integridad y confidencialidad de la información.



```
1 -- 1. tipo de dato para los roles del equipo
2 CREATE TYPE rol_usuario AS ENUM ('analista_junior', 'analista_senior', 'administrador');
3
4 -- 2. tabla de perfiles vinculada a la autenticación nativa de Supabase
5 CREATE TABLE IF NOT EXISTS perfiles_usuarios (
6   id UUID REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY,
7   correo TEXT NOT NULL,
8   rol rol_usuario DEFAULT 'analista_junior'::rol_usuario,
9   fecha_creacion TIMESTAMPTZ WITH TIME ZONE DEFAULT NOW()
10 );
11
12 -- 3. Habilitar la seguridad a nivel de fila (RLS) en tu tabla principal
13 ALTER TABLE transacciones ENABLE ROW LEVEL SECURITY;
14
15 -- 4. POLÍTICA: Administradores y Senior pueden ver y modificar TODAS las transacciones
16 CREATE POLICY "Acceso total para Senior y Admin"
```

Results Chart Export

Source Primary Database Role postgres Run

Success. No rows returned

### EVIDENCIA DE DATOS CARGADOS EN LA BASE DE DATOS

La captura muestra la tabla **transacciones** en el Table Editor de Supabase tras la ejecución exitosa del pipeline de datos. Esta evidencia técnica confirma que los procesos de transformación (limpieza de duplicados, imputación de nulos y cálculo de flags de anomalías) se ejecutaron correctamente en el entorno de desarrollo

Table Editor

public.transacciones

Filter by id\_transaccion, id\_cliente, fecha\_hora... or ask AI

|       | id_transaccion | id_cliente | fecha_hora          | monto_usd | tipo_comercio | estado    |
|-------|----------------|------------|---------------------|-----------|---------------|-----------|
| T-001 |                | C-101      | 2026-06-18 08:00:00 | 50.0      | nacional      | aprobada  |
| T-002 |                | C-102      | 2026-06-18 08:15:00 | 0.0       | internacional | rechazada |
| T-003 |                | C-103      | 2026-06-18 08:20:00 | 120.5     | nacional      | pendiente |
| T-004 |                | C-104      | 2026-06-18 08:45:00 | 1850.0    | internacional | aprobada  |
| T-005 |                | C-101      | 2026-06-18 09:30:00 | 350.0     | internacional | aprobada  |
| T-006 |                | C-105      | 2026-06-18 09:35:00 | 0.0       | nacional      | rechazada |
| T-007 |                | C-102      | 2026-06-18 09:40:00 | 25.0      | nacional      | aprobada  |
| T-008 |                | C-102      | 2026-06-18 09:45:00 | 500.0     | internacional | rechazada |
| T-009 |                | C-106      | 2026-06-18 09:50:00 | 2000.0    | nacional      | aprobada  |
| T-010 |                | C-107      | 2026-06-18 09:55:00 | 1600.0    | internacional | pendiente |
| T-011 |                | C-108      | 2026-06-18 10:00:00 | 45.0      | nacional      | aprobada  |
| T-012 |                | C-109      | 2026-06-18 10:05:00 | 80.0      | internacional | aprobada  |
| T-013 |                | C-110      | 2026-06-18 10:10:00 | 210.0     | nacional      | aprobada  |
| T-014 |                | C-111      | 2026-06-18 10:15:00 | 15.5      | nacional      | aprobada  |

Page 1 of 2 100 rows 122 records

### EVIDENCIAS DE CONSULTA FUNCIONAL EN SUPABASE

La ejecución de la consulta SQL demuestra la capacidad de implementar lógica de negocio avanzada directamente en la capa de datos.

Untitled query

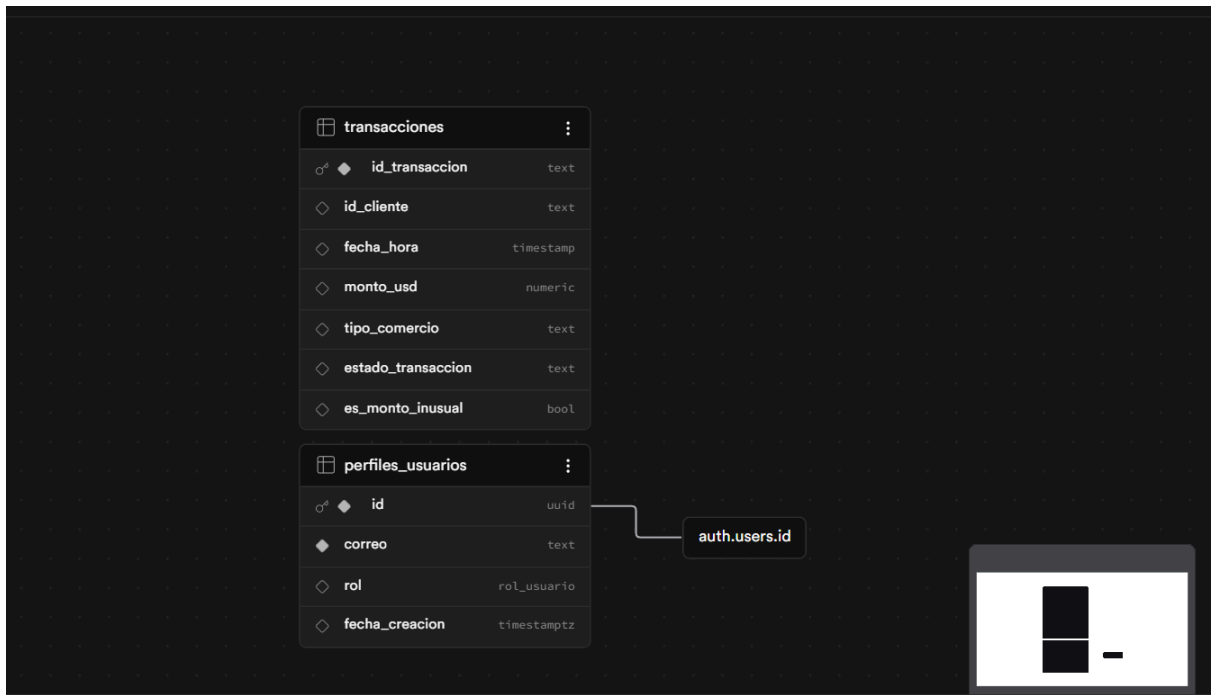
```
1
2 --uso de cte
3 WITH transacciones_aprobadas AS
4     SELECT
5         id_cliente,
6         id_transaccion,
7         fecha_hora,
8         monto_usd,
9         estado_transaccion,
10        --uso de window function
11        LAG(monto_usd) OVER (PARTITION BY id_cliente ORDER BY fecha_hora) AS monto_anterior
12    FROM transacciones
13    --condicion que sea aprobada
14    WHERE estado_transaccion = 'aprobada'
15
16 SELECT
```

Results Chart Export

| id_cliente | id_transaccion | fecha_hora          | estado_trans | monto_anterior | monto_actual |
|------------|----------------|---------------------|--------------|----------------|--------------|
| C-101      | T-005          | 2026-06-18 09:30:00 | aprobada     | 50.0           | 350.0        |
| C-101      | T-117          | 2026-06-18 19:15:00 | aprobada     | 10.0           | 80.0         |
| C-120      | T-024          | 2026-06-18 11:15:00 | aprobada     | 25.0           | 150.0        |
| C-146      | T-051          | 2026-06-18 13:50:00 | aprobada     | 15.0           | 100.0        |

### VISUALIZADOR DE ESQUEMA:

El diagrama de esquema (Schema Visualizer) ilustra la integridad referencial y la arquitectura de seguridad que sustenta el proyecto. Se observa la relación clave entre la tabla `transacciones` y la tabla `perfiles_usuarios`, la cual está vinculada directamente al sistema de autenticación nativo de Supabase (`auth.users.id`).



## CÓDIGO DE CREACIÓN DE ROLES:

```
--CREACION DE ROLES PARA USUARIO DE SUPABASE

-- 1 ENUM para los roles del equipo
CREATE TYPE rol_usuario AS ENUM ('analista_junior', 'analista_senior',
'administrador');

-- 2. tabla de perfiles vinculada a la autenticación nativa de Supabase
CREATE TABLE IF NOT EXISTS perfiles_usuarios (
    id UUID REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY,
    correo TEXT NOT NULL,
    rol rol_usuario DEFAULT 'analista_junior'::rol_usuario,
    fecha_creacion TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 3. Habilitar la seguridad a nivel de fila (RLS) en tu tabla
principal
ALTER TABLE transacciones ENABLE ROW LEVEL SECURITY;
```

```

-- 4. POLÍTICA 1: Administradores y Senior pueden ver y modificar TODAS
las transacciones
CREATE POLICY "Acceso total para Senior y Admin"
ON transacciones
FOR ALL
TO authenticated
USING (
    EXISTS (
        SELECT 1 FROM perfiles_usuarios
        WHERE perfiles_usuarios.id = auth.uid()
        AND perfiles_usuarios.rol IN ('analista_senior'::rol_usuario,
'administrador'::rol_usuario)
    )
);

-- 5. POLÍTICA 2: Analistas Junior SOLO pueden leer las transacciones
que estén 'aprobadas'
CREATE POLICY "Lectura restringida para Junior"
ON transacciones
FOR SELECT
TO authenticated
USING (
    estado_transaccion = 'aprobada'
    AND EXISTS (
        SELECT 1 FROM perfiles_usuarios
        WHERE perfiles_usuarios.id = auth.uid()
        AND perfiles_usuarios.rol = 'analista_junior'::rol_usuario
    )
);

```

### **CÓDIGO SQL DE CONSULTA:**

```

--CONSULTA SQL PARA OBTENER TRANSACCIONES APROBADAS CON UN MONTO ACTUAL
5 VECES MAYOR AL ANTERIOR
--uso de cte
WITH transacciones_aprobadas AS (
    SELECT
        id_cliente,

```

```

        id_transaccion,
        fecha_hora,
        monto_usd,
        estado_transaccion,
        --uso de window function
        LAG(monto_usd) OVER (PARTITION BY id_cliente ORDER BY
fecha_hora) AS monto_anterior
    FROM transacciones
    --condicion que sea aprobada
    WHERE estado_transaccion = 'aprobada'
)
SELECT
    id_cliente,
    id_transaccion,
    fecha_hora,
    estado_transaccion,
    monto_anterior,
    monto_usd AS monto_actual
FROM transacciones_aprobadas
-- filtro estricto de las 5 veces
WHERE monto_anterior IS NOT NULL
    AND monto_usd >= (monto_anterior * 5);

```

## Análisis de Resultados y Conclusiones

Tras ejecutar la consulta de detección de anomalías, el sistema ha identificado satisfactoriamente los "saltos bruscos" de dinero, filtrando únicamente las transacciones aprobadas.

### 1. Hallazgos del Pipeline

Los resultados obtenidos muestran transacciones que superan en 5 veces el monto de la transacción anterior del mismo cliente. Por ejemplo:

- **Cliente C-101 (T-005):** Presentó un incremento de \$50.0 a \$350.0, lo cual activa nuestra alerta de seguridad.
- **Cliente C-120 (T-024):** Presentó un incremento de \$25.0 a \$150.0, marcando un comportamiento de gasto atípico.

### 2. Interpretación de Negocio

Este pipeline aporta valor real al área de Prevención de Fraude mediante:

- **Automatización del monitoreo:** El equipo de analistas ya no necesita revisar logs manualmente; el sistema entrega una lista filtrada de riesgos diariamente.

- **Reducción de Falsos Positivos:** Al aislar solo las transacciones **aprobadas** (Regla 4), garantizamos que las alertas reportadas representan **movimientos financieros reales** y no intentos de fraude fallidos (rechazados), lo que permite al equipo de fraude actuar con mayor precisión y eficiencia.
- **Escalabilidad:** Aunque este MVP se probó con un archivo pequeño, la lógica aplicada (CTEs y Window Functions) es capaz de escalar a millones de transacciones sin necesidad de reescribir la lógica de negocio.

### 3. Toma de Decisiones y Conclusión Final

La arquitectura construida demuestra que es posible pasar de datos "crudos" y con ruido a *insights* de valor de manera automática.

La principal decisión técnica probado ser correcta para este alcance, permitiendo una ejecución de consultas analíticas inmediata. Con la integración de **Apache Airflow**, este proceso deja de ser un script aislado y se convierte en un **servicio de monitoreo activo** que opera sin intervención humana, cumpliendo con la criticidad que el sector bancario demanda.